

NearSave (ReWorth) — Ultimate Interview Guide

Every Single Interview Question & Answer in Simple, Clear English

Project: NearSave (Hyperlocal Expiry Marketplace)
Architecture: Modular Monolith Layered Architecture
Backend: Java 17/21/26, Spring Boot 3.2.0, Hibernate

Database: MySQL 8 + Spatial Indexing (JTS)
Security: Spring Security 6, Stateless JWT, BCrypt
Frontend: Semantic HTML5, Vanilla ES6 JS, Inter CSS

1. Project Overview & Business Logic

Questions about the domain, product rules, and user flow

Q1: What is NearSave? Give a 1-minute elevator pitch.

NearSave is a hyperlocal marketplace web platform that helps grocery retailers sell products approaching their expiry date to nearby customers at heavy discounts.

It solves two problems: it prevents grocery store inventory waste and helps price-sensitive consumers save money on everyday groceries.

Retailers list near-expiry products with automated 40% to 60% discounts. Customers discover items within 1 km via GPS, reserve a pickup token for 30 minutes, and pay directly at the shop when collecting the goods.

Key Takeaway for Interviewer: State problem, solution, users, business rules, and technical stack clearly in 3-4 sentences.

Q2: What is the Core Product Rule in NearSave?

NearSave enforces two critical business constraints to maintain quality and safety:

- **Expiry Safety Window:** A retailer cannot list any product unless its expiry date is at least 30 days into the future. This prevents spoiled or unsafe food from ever being listed.
- **Controlled Discount Range:** The discounted price must strictly fall between 40% and 60% of the MRP (Maximum Retail Price). This guarantees real savings for customers and fair revenue for retailers.

Q3: What are the three User Roles and what can each role do?

1. **CUSTOMER:** Discovers nearby discounted products via GPS, filters by category/price, books 30-minute pickup tokens, views active/past tokens, and cancels reservations.
2. **RETAILER:** Registers and manages their physical shop, lists products with automated discount calculations, tracks active reservations, and verifies customer 6-character tokens at the counter.
3. **ADMIN:** Monitors platform-wide analytics (revenue, items sold, expiration rate), approves new retailer shop registrations, and inspects real-time audit logs.

Q4: What is the step-by-step customer journey from discovery to pickup?

Step 1: The customer visits the homepage; the browser detects their GPS location.

Step 2: The backend returns all active, discounted products within a 1 km radius.

Step 3: Customer clicks 'Book' on an item. A 30-minute reservation token (e.g. 'NS-4982') is generated, and stock is decremented by 1.

Step 4: The customer walks to the store and shows the 6-character code to the retailer.

Step 5: The retailer types the code into their dashboard verify box. The backend marks the token COMPLETED, records a pickup event, and the customer pays cash/UPI in person.

2. Architecture & Technical Decisions

Modular monolith, package structure, and tech stack choices

Q5: Why did you choose a Modular Monolith instead of Microservices?

- Transactional Atomicity: Booking an item requires locking stock, validating expiration, and creating a reservation record in one single database transaction. Doing this in microservices requires complex Sagas or two-phase commits.
- Zero Network Latency: In a monolith, method calls between services happen in-memory rather than over HTTP/gRPC.
- Simple Deployment & Lower Cost: NearSave can be deployed as a single lightweight JAR container on cloud platforms like Render or AWS EC2 without managing Kubernetes clusters.

Key Takeaway for Interviewer: Acknowledge trade-offs. Choosing a modular monolith for this scale shows maturity and cost consciousness.

Q6: How is the Java package structure organized?

We organize code by Feature (Package-by-Feature) rather than traditional Layer (Package-by-Layer):

- com.nearsave.auth: Controllers, JWT filter, token utilities, and login logic.
- com.nearsave.product: Product entity, spatial repository queries, and explore service.
- com.nearsave.reservation: Booking entity, state machine, 30-min timer, and cron scheduler.
- com.nearsave.shop: Shop entity, retailer profile, and approval workflows.
- com.nearsave.admin: Platform audit events and admin review APIs.
- com.nearsave.analytics: Aggregated metrics calculation for retailers and admins.
- com.nearsave.common: Global exception handler, API responses, and database seeder.

Q7: Why did you choose Spring Boot 3 over other frameworks?

- Production-Ready: Built-in HikariCP connection pool, embedded Tomcat server, and Actuator metrics.
- Modern Java Standards: Spring Boot 3 requires Java 17+, providing access to record classes, pattern matching, and superior garbage collectors (ZGC/G1).
- Seamless Spatial Integration: Hibernate 6 in Spring Boot 3 supports Spatial/GIS types natively without deprecated custom dialects.

3. Core Java & OOP Concepts in NearSave

Java fundamentals, precision math, collections, and concurrency

Q8: Why did you use BigDecimal instead of double or float for prices?

Float and double in Java use binary floating-point arithmetic (IEEE 754 standard). They cannot accurately represent decimal fractions like 0.1 or 0.7, leading to rounding errors (e.g., $0.1 + 0.2 = 0.30000000000000004$).

In an e-commerce platform, rounding errors can cause money discrepancies in revenue and MRP discount checks. BigDecimal provides arbitrary precision and exact mathematical representation with explicit rounding modes (e.g. RoundingMode.HALF_UP).

Key Takeaway for Interviewer: Always use BigDecimal for currency and financial calculations in enterprise applications.

Q9: How did you use Object-Oriented Programming (OOP) in this project?

- Encapsulation: Entities like Product and User have private fields with controlled getters/setters via Lombok. Status changes are guarded by validation methods.
- Abstraction: Controllers depend on Service interfaces, and Services depend on JpaRepository interfaces. Callers don't know whether data comes from MySQL, memory, or cache.
- Polymorphism: Custom runtime exceptions (AppException) inherit from RuntimeException and are handled polymorphically by @ExceptionHandler in GlobalExceptionHandler.

Q10: What Java Collections did you use and why?

- List (ArrayList): For returning ordered query results such as recent reservations and nearby products.
- Set (HashSet): In js/products.js and backend filters to quickly calculate unique nearby shops without duplicates in O(1) time.
- Stream API: Used .filter(), .map(), and .collect(Collectors.toList()) to transform JPA entity objects into lightweight ProductResponse DTOs cleanly.

4. Database Design, Spatial Indexing & JTS

MySQL 8 spatial functions, coordinates, and constraints

Q11: How does Hyperlocal Proximity Search work under the hood?

1. Each shop's geographic coordinate is stored in MySQL as a POINT column with SRID 4326.
2. When a customer opens the site, their browser GPS coordinates (lat, lng) are sent as query parameters.
3. We use the native MySQL function ST_Distance_Sphere():
`ST_Distance_Sphere(p.shop_location, ST_SRID(POINT(:userLat, :userLng), 4326)) <= 1000`
4. ST_Distance_Sphere calculates the spherical distance in meters directly on the Earth's surface.
5. A SPATIAL INDEX on shop_location uses an R-Tree data structure to filter bounding boxes in O(log N) time rather than scanning every row.

Q12: What is the coordinate order difference between JTS and MySQL POINT?

- JTS (Java Topology Suite): Points are constructed with `Coordinate(x, y) = new Coordinate(longitude, latitude)`.
- MySQL POINT: The SQL function `POINT(x, y)` expects `POINT(latitude, longitude)` when mapped with geographic SRID 4326.

If this order is inverted, coordinates end up swapped, placing shops in wrong continents. We verified this alignment so nearby searches return accurate meter distances.

Q13: What database constraints protect data integrity in NearSave?

- CHECK (stock_quantity >= 0): Guarantees inventory cannot drop below zero even under concurrency.
- UNIQUE (token_code): Ensures every 6-character reservation token is globally unique.
- UNIQUE (user_id, target_id, target_type): Prevents duplicate customer favorites.
- Foreign Keys with ON DELETE RESTRICT: Prevents accidental deletion of a shop or user if active reservations exist.

5. Security, JWT & Role-Based Access Control

Token lifecycle, filters, BCrypt, and endpoint protection

Q14: Explain the step-by-step JWT authentication flow in NearSave.

1. Login: Client sends POST /api/v1/auth/login with email and password.
2. Verification: DaoAuthenticationProvider checks password using BCryptPasswordEncoder.
3. Token Generation: JwtUtil signs an HMAC-SHA256 token containing claims (email, role, userId, 24-hour expiration).
4. Storage: Frontend stores JWT in localStorage and user profile in sessionStorage.
5. Interception: On every API call, JwtAuthFilter intercepts the Authorization: Bearer header.
6. SecurityContext: If valid, a UsernamePasswordAuthenticationToken is placed in Spring's SecurityContextHolder.
7. Authorization: @PreAuthorize("hasRole('RETAILER')") checks if the authenticated user has permission.

Q15: How is password security handled? Why BCrypt?

We use BCryptPasswordEncoder with a salt work factor of 10.

- Why BCrypt? It includes a randomly generated cryptographic salt automatically with each hash, protecting against Rainbow Table attacks.
- Slow by Design: BCrypt is intentionally computationally expensive, making brute-force and dictionary attacks impractical. Passwords are never stored in plaintext.

Q16: How do you prevent a Customer from calling Retailer or Admin APIs?

We use two security layers:

1. HttpSecurity Filter Chain: In SecurityConfig.java, we declare .requestMatchers("/api/v1/admin/**").hasRole("ADMIN") and .requestMatchers("/api/v1/products").hasRole("RETAILER").
2. Method-Level Security: We annotate controllers with @PreAuthorize("hasRole('RETAILER')"). If a Customer JWT attempts to access these endpoints, Spring Security throws an AccessDeniedException and returns HTTP 403 Forbidden.

6. Concurrency, Locking & State Machine

Pessimistic locking, race conditions, and transition guards

Q17: Explain how Pessimistic Locking prevents double-booking.

Problem: If 2 users click 'Book' on the last item simultaneously, both read stock=1 and both decrement to 0, causing 2 reservations for 1 physical product (Race Condition).

Solution: We use Pessimistic Write Locking (@Lock(LockModeType.PESSIMISTIC_WRITE)):

1. ProductRepository executes `SELECT * FROM products WHERE id = :id FOR UPDATE`.
2. MySQL locks that specific row. Any other concurrent request must wait.
3. The first request validates `stock > 0`, decrements stock to 0, saves the reservation, and commits.
4. When the lock releases, the second request reads the updated row, sees `stock == 0`, and throws a 409 Conflict exception.

```
@Lock(LockModeType.PESSIMISTIC_WRITE) @Query("SELECT p FROM Product p WHERE p.id = :id") Optional
findByIdWithLock(@Param("id") Long id);
```

Key Takeaway for Interviewer: Pessimistic locking is ideal for inventory checkout with high contention because it prevents costly optimistic lock retry storms.

Q18: What is the difference between Pessimistic Locking and Optimistic Locking?

- Optimistic Locking: Uses a @Version number column. It does not lock the database row. When committing, it checks if the version changed. If yes, it throws OptimisticLockException. Best for read-heavy systems with low write contention.
- Pessimistic Locking: Uses database-level row locks (`SELECT ... FOR UPDATE`). It blocks other transactions from reading or writing until the current transaction commits. Best for hot-spot inventory and ticket booking where contention is high.

Q19: Explain the Reservation State Machine and its transition validations.

A reservation token moves through explicit lifecycle states:

- RESERVED -> READY_FOR_PICKUP: Retailer prepares the item.
- READY_FOR_PICKUP -> COMPLETED: Retailer verifies token code upon customer arrival.
- RESERVED / READY_FOR_PICKUP -> CANCELLED: Customer cancels; stock is restored.
- RESERVED / READY_FOR_PICKUP -> EXPIRED: 30-min timer expires; scheduler restores stock.
- READY_FOR_PICKUP -> NO_SHOW: Customer never showed up; slot released.

Invalid jumps (e.g. COMPLETED -> CANCELLED) are blocked with an `IllegalStateException`.

7. Background Schedulers & Transaction Propagation

Cron jobs, inventory recovery, and Propagation.REQUIRES_NEW

Q20: How does the background token expiry scheduler work?

NearSave uses Spring's @EnableScheduling with a cron trigger:

```
@Scheduled(cron = "0 * * * *") // Runs at second 0 of every minute
```

1. Queries for reservations with status IN ('RESERVED', 'READY_FOR_PICKUP') and `expires_at < NOW()`.
2. Iterates over expired records, marks status = EXPIRED.
3. Locks the corresponding product and increments `stock_quantity` by 1.
4. Logs an automated audit event so the shopkeeper can see why stock was returned.

Q21: Why do you use `Propagation.REQUIRES_NEW` in `AuditService`?

In Spring's `@Transactional` system, the default propagation is `REQUIRED` (joins the caller's transaction).

If a business operation fails (e.g. invalid payment or out-of-stock) and rolls back, any audit logs written in that transaction would also roll back and disappear!

By setting `propagation = Propagation.REQUIRES_NEW`, Spring suspends the caller's transaction and executes the audit insert in a completely separate, independent transaction.

The audit event is committed to the database even if the parent business transaction fails.

```
@Transactional(propagation = Propagation.REQUIRES_NEW) public void logEvent(String actor, String action, String entityName, Long entityId, String metadata) { auditEventRepository.save(new AuditEvent(...)); }
```

8. Testing Suite & 100-Thread Concurrency Benchmark

How the concurrency test was built and passed

Q22: How did you test your concurrency control? Explain the 100-thread test.

We built an automated integration test in `NearSaveConcurrencyTest.java`:

1. Setup: Created 1 product with `stockQuantity = 1` and seeded 100 customer users.
2. Latch: Created a `CountDownLatch(1)` and a thread pool with `Executors.newFixedThreadPool(100)`.
3. Simultaneous Burst: All 100 threads waited on `latch.await()`. Once `latch.countDown()` fired, all 100 threads called `reservationService.reserveProduct()` at the exact same millisecond.
4. Verification: We used `AtomicInteger` counters to verify that exactly 1 request succeeded and 99 failed with `AppException/Conflict`. We then asserted that the final database stock was exactly 0.

Key Takeaway for Interviewer: Using `CountDownLatch` and `AtomicInteger` demonstrates practical multi-threading expertise in Java.

Q23: What HikariCP issue happened during testing and how did you resolve it?

Problem: The test failed with `HikariPool-1 - Connection is not available, request timed out after 30000ms`. Why? HikariCP defaults to `maximum-pool-size = 10`. When 100 threads held row locks and waited on transactions, all 10 connections were locked, causing pool starvation deadlocks.

Solution: We created `application-test.properties` under the test profile with `spring.datasource.hikari.maximum-pool-size = 110`. This allowed all concurrent threads to acquire connections smoothly without starving.

9. Real Technical Challenges & Debugging (STAR Method)

Impressive answers to 'Tell me about a challenge you solved'

Q24: Challenge 1: MySQL Spatial SRID Mismatch (ER_GIS_DIFFERENT_SRIDS)

- Situation: The nearby product search crashed with a database error: ER_GIS_DIFFERENT_SRIDS.
- Task: Find why MySQL rejected the spatial distance query.
- Action: I discovered that shop coordinates were saved with SRID 4326 (WGS 84 GPS), but incoming user coordinates were constructed using POINT(:lat, :lng) without an SRID, defaulting to SRID 0. MySQL 8 refuses to calculate distance between different spatial reference systems. I fixed the query to explicitly cast: ST_SRID(POINT(:userLat, :userLng), 4326).
- Result: The query executed in under 5 milliseconds with accurate distance calculation.

Q25: Challenge 2: MySQL ENUM Column Truncation on Database Migration

- Situation: When introducing the ADMIN role, user authentication failed with Data truncated for column 'role' at row 1.
- Task: Enable the ADMIN role without dropping existing user data.
- Action: The original database schema had created the column as ENUM('RETAILER', 'CUSTOMER'). In MySQL, Hibernate ddl-auto=update cannot modify enum sets. I wrote an automated DatabaseSeeder that executes ALTER TABLE users MODIFY COLUMN role VARCHAR(50) NOT NULL safely on application startup.
- Result: The column was migrated to VARCHAR(50), allowing new roles without schema conflicts.

Q26: Challenge 3: Disappearing Products Bug (Expiry Filter Logic)

- Situation: Retailers listed products with 31 days expiry, but after listing, products were not showing up for customers.
- Task: Identify why unexpired products were filtered out of nearby discovery.
- Action: The SQL query had an incorrect condition: AND p.expiry_date > DATE_ADD(NOW(), INTERVAL 1 MONTH). This meant products were only visible if they expired in MORE than 30 days! As soon as an item approached expiry (the core purpose of NearSave), it disappeared! I changed the query to p.expiry_date >= DATE(NOW()).
- Result: All unexpired discounted items remain visible until their actual expiry date.

Q27: Challenge 4: Browser Geolocation Testing Fallback

- Situation: During local testing, developers outside Bhopal could not see any products because the test shop was seeded in Bhopal, exceeding the 1 km limit.
- Task: Enable developers to test the UI from any location while preserving the 1 km hyperlocal rule in production.
- Action: In ProductService.getNearbyProducts, I added a smart demo fallback: if 0 products are found within 1 km, the search radius expands to 10,000 km while still calculating and displaying the real calculated distance on the card (e.g. '1,200 km away').
- Result: Testers can immediately verify product cards from any browser location.

10. Scenario-Based Questions & Future Scaling

System design, failure scenarios, and scalability enhancements

Q28: What if the customer's phone clock is 10 minutes fast? Does their token expire early?

No. The frontend timer is strictly cosmetic for user experience.

Authoritative time is enforced exclusively by the backend server database timestamp (expires_at in UTC).

Even if the customer manipulates their phone clock or refreshes the page, the retailer's verification endpoint checks against the database clock, preventing client-side tampering.

Q29: How would you scale NearSave to support 500,000 active users?

1. Redis Geospatial Caching: Cache shop locations in Redis using GEOADD and GEORADIUS to eliminate spatial database queries on high-traffic browse endpoints.
2. Read/Write Split: Deploy MySQL Master-Replica architecture. Route read-only search/explore queries to replicas and booking transactions to the primary.
3. WebSockets / SSE: Replace 60-second polling on customer tokens with WebSocket events for real-time pickup status notifications.
4. CDN Caching: Cache static UI assets (CSS, JS, icons) on Cloudflare or AWS CloudFront edge servers.

Q30: Why should we hire you based on your work in NearSave?

- Full Lifecycle Engineering: I designed, implemented, debugged, and benchmarked an end-to-end commercial system rather than a trivial student tutorial.
- Concurrency & Lock Mastery: I understand row-level locking, race conditions, connection pool sizing, and how to verify them with multi-threaded tests.
- Hands-on Debugging Skills: I resolved real production obstacles including spatial SRID mismatches, database enum migrations, and HikariCP connection starvation.
- Clean Architecture: I organized the codebase into modular feature packages with transaction isolation, clean REST APIs, and comprehensive documentation.